ChatGPT

Fine-Tuning TinyLlama for a Safe, Empathetic Mental Health Companion

Data Curation: Collect and clean diverse mental-health conversational data. Use anonymized therapy transcripts or counseling Q&A from real dialogues (e.g. the *"mental_health_counseling_conversations"* dataset on Hugging Face [1]). Supplement this with CBT/SFBT prompt–response pairs or peer-support chats. Include structured assessments (PHQ-9, GAD-7, etc.) by framing them as user-provided scores or histories so the model can tailor its tone and suggestions. Ensure all data is de-identified and balanced across issues (depression, anxiety, trauma, etc.). For multilingual support, incorporate translated transcripts or non-English therapy logs into the training mix. *Example:* A dataset entry might begin with a system prompt like "You are a helpful mental health counseling assistant, please answer based on the patient's description..." [2] [1].

- **Sources:** Use public therapy Q&As (e.g. Amod's counseling dataset [1]) and synthesize instruction pairs ("User: *feeling worthless...*, Assistant: *I'm sorry you feel this way...*").
- **Preprocessing:** Clean slang/typos, remove personal identifiers, and ensure examples reflect supportive, evidence-based therapy. Balance the dataset for various demographics and languages if multilingual functionality is desired.

Instruction Tuning & Training Configs

Fine-tune TinyLlama using parameter-efficient techniques on the curated data. Load the base TinyLlama model with quantization to fit Kaggle's GPU: for example, use 4-bit or 8-bit quantization via BitsAndBytes/Bf16 (as shown in HuggingFace examples) to drastically reduce VRAM needs [3]. Then attach a LoRA adapter (a low-rank projection) so that only a small fraction of weights are updated during training [4]. This lets a 1.1B model train on a single 16GB GPU (e.g. Tesla P100) by requiring far less memory and compute. A typical setup might be: one epoch over the data, per-device batch size = 1 or 2 with gradient accumulation, learning rate ~1–5e-4, warmup steps ~10, and no mixed precision if stability is an issue [5]. Use the Hugging Face `Trainer` or `SFTTrainer` from the TRL library to handle dialogue formatting and optimization. For example, you can format each example with a system prompt (e.g. *"You are a counseling assistant..."* [6]), the user query, and the target therapist response. Then train with `TrainingArguments` similar to:

```
per_device_train_batch_size=1, num_train_epochs=3, learning_rate=2e-4,
gradient_accumulation_steps=2, fp16=False
```

(as illustrated in practice [5]). Use low dropout and weight decay to stabilize fine-tuning on small data.

- **Techniques:** Employ Hugging Face's PEFT library (LoRA) and BitsAndBytes quantization. Optionally experiment with QLoRA (quantized LoRA) if extra VRAM reduction is needed.

- **Data Augmentation:** For multilingual support, either fine-tune separate LoRA weights on different language subsets, or include translated examples in one mix. You can also prompt-translate user input at inference time.

Safety Alignment and Ethical Guardrails

After supervised fine-tuning, apply an alignment step (akin to RLHF) to enforce helpfulness and safety. Implement a reward model or rule-based filter that penalizes or blocks harmful replies. For example, follow OpenAI's approach of training with human feedback: have annotators (or simulation heuristics) rank model outputs and fine-tune to prefer those that are empathic and non-toxic [7]. More simply, run a post-filter on generated text to detect disallowed content (self-harm, profanity, illegal advice, explicit content, etc.). Tools include keyword/blocklist filters or classifier models (e.g. HuggingFace's moderation transformers). Critically, any mention of suicide, self-harm, abuse, or similar triggers a safe completion: the model should not elaborate freely but instead provide a supportive response with a referral. For instance, auto-detect phrases like "kill myself" and respond with empathetic defusion and a hotline: *"I'm so sorry you feel like this. It might help to reach out to a mental health professional or crisis line…"* with links or numbers. Character.AI did something similar by popping up suicide-prevention resources when self-harm is mentioned [8].

[8] [7] demonstrate the need for such measures: experts warn that AI chatbots are "unqualified to discuss" complex self-harm topics without safeguards [9]. In practice, combine methods: a trained safety classifier plus hand-crafted rules. Clearly flag the bot as non-human (avoid false authenticity), never provide medical diagnoses or prescriptions, and include disclaimers encouraging professional help.

- **Example Safety Measures:**
- Suicide/self-harm detection → emergency response (hotline link) [8].
- Prevent giving legal/medical advice (e.g. responses: "I'm not qualified to advise, but…").
- Reject any hate speech or encouragement of dangerous acts.

Memory and Context Continuity

Implement conversation memory so the bot feels like a continuous therapist. Short-term memory comes from including the recent dialogue context (last few user–assistant turns) in each prompt. For longer-term memory, use summarization or retrieval. For instance, LangChain's **ConversationSummaryMemory** progressively condenses the chat into a running summary (via an LLM) and feeds that summary back in as `{history}` [10]. This way, early-session details (user's background, progress) aren't lost even if the token window fills. As [38†L489-L494] shows, summarizing allows "much longer conversations" by keeping the token count from growing linearly [10]. Alternatively, store key user facts in a vector database and retrieve them as needed (RAG-style). Memory can hold user profile info (age, stressors, PHQ-9 score) or personal goals. Ensure any memory is private/on-device.

- **Memory Workflow:** At each turn, append the last 2–3 turns to context. Every N turns (or per message), run a summarization LLM to update a "chat summary". Prepend this summary into the prompt for future replies [10].
- **Long-term Memory:** Optionally log important user preferences or coping strategies (with consent) and remind the user.

Empathy and Conversational Psychologist Style

To make the chatbot truly “psychologist-like”, fine-tune it on dialogue that exhibits empathy and active listening. Encourage output with validating language (“I understand…”, “That sounds really hard…”), open questions, and step-by-step guidance. The training examples (from CBT or counseling transcripts) should model this style. Additionally, equip the bot with task-like capabilities: it can offer coping **strategies** (deep breathing, cognitive reframing, mindfulness exercises) and psychoeducation on issues. For example, the Savience healthcare blog notes that mental-health chatbots should “engage in empathetic conversations [and] offer coping strategies for stress and anxiety” while acting as a non-judgmental outlet [11]. In practice, this means programming the assistant to ask clarifying questions (“Can you tell me more about what happened?”), summarize user feelings, and suggest interventions (e.g. “When you feel anxious, try a grounding exercise like five deep breaths”).

Additional Capabilities: The model can do more than text chat:
- *Mood/Journaling:* Prompt the user to log feelings or done homework.
- *Reminders:* Check in on coping goals (e.g. “How was your mindfulness practice yesterday?”).
- *Resource Suggestions:* Provide links to vetted articles or hotlines if user requests.
- *Crisis Path:* If the user mentions self-harm, automatically shift to a safety protocol as in [30†L189-L193].

These features turn a generic chatbot into a “therapeutic companion” that not only answers but actively guides the user with evidence-based techniques.

Deployment and On-Device Serving

After fine-tuning, quantize the model for deployment on constrained hardware. Convert the TinyLlama weights to INT8 (or INT4) using tools like llama.cpp or HuggingFace’s `transformers` quantization. In testing, Raspberry Pi 5 (8GB) can run a 1–3B model at usable speeds [12]. For example, benchmarks show TinyLlama (~1.1B) generates ~5–7 tokens/second on a Pi5 CPU [12]. For serving, use a lightweight runtime. Ollama (an easy-to-install local LLM server) supports ARM64: on Pi5 install the ARM64 build via `curl https://ollama.com/install.sh | sh` [13]. Then run the quantized chat model with Ollama (e.g. `ollama run tinyllama-chat`). This requires no internet and keeps user data local. Ensure to disable any logging of conversations for privacy.

- **Quantization & Performance:** Pick a GGUF format (e.g. Q4_K_M) for a balance of size and speed. TinyLlama 1.1B in Q4_K quant runs in ~5–7 tps [12]. Keep word limits low (maybe 1024 tokens) to stay responsive.
- **Ollama Serving:** Ollama auto-detects the GPU; on Pi it will run CPU-only, which is fine. As [43†L293-L300] shows, install with `curl` and verify with `ollama --version`.

Continuous Evaluation and Updates

Before public use, evaluate the model’s outputs on key criteria: **empathy** (do human judges find it caring?), **relevance/coherence** (answers fit the question), and **safety** (no disallowed content). Use a mix of automated checks (e.g. safety classifiers) and human review. After deployment, collect (opt-in) user feedback and conversation logs to spot failure modes. Periodically re-fine-tune on new data (while retraining the safety filters) to improve performance. This closed loop ensures the bot stays up-to-date and mitigates drift or unexpected outputs over time.

Outcome: The result is a private, on-device assistant that speaks like a therapist: empathic, multilingual, and cautious. It draws on therapy-informed training to give helpful coping advice, uses memory to maintain continuity, and enforces ethical guardrails so it never encourages harm. Such a model augments — but does not replace — human care, acting as a supportive companion available anytime. [11] [12]

Sources: We based this design on recent work and tools: e.g., Hugging Face examples of LoRA fine-tuning with 4-bit quantization [3] [4], clinical chatbot reviews [14], OpenAI's alignment research [7], and engineering guides for on-device LLMs [12] [13]. These underline the need for strong safety measures and efficient training when building mental-health AI.

---

[1] Amod/mental_health_counseling_conversations · Datasets at Hugging Face
https://huggingface.co/datasets/Amod/mental_health_counseling_conversations

[2] fadodr/mental_health_therapy · Datasets at Hugging Face
https://huggingface.co/datasets/fadodr/mental_health_therapy

[3] [4] [5] [6] LLaMA 4 Fine-Tuning with Mental Health Counseling Data
https://huggingface.co/blog/ImranzamanML/llama-4-fine-tuning-with-mental-health-counseling

[7] Aligning language models to follow instructions | OpenAI
https://openai.com/index/instruction-following/

[8] [9] CNN: Senators demand information from AI companion apps following kids' safety concerns, lawsuits - Senator Alex Padilla
https://www.padilla.senate.gov/newsroom/news-coverage/cnn-senators-demand-information-from-ai-companion-apps-following-kids-safety-concerns-lawsuits/

[10] Conversational Memory for LLMs with Langchain | Pinecone
https://www.pinecone.io/learn/series/langchain/langchain-conversational-memory/

[11] The Role of AI Chatbots in Healthcare Communications
https://savience.com/2024/07/08/the-role-of-ai-chatbots-in-healthcare-communications/

[12] Running Llama On Raspberry Pi 5 (2025 Setup & Guide)
https://aicompetence.org/running-llama-on-raspberry-pi-5/

[13] Running DeepSeek R1 Locally on a Raspberry Pi - DEV Community
https://dev.to/jeremycmorgan/running-deepseek-r1-locally-on-a-raspberry-pi-1gh8

[14] (PDF) Improving Workplace Well-being in Modern Organizations: A Review of Large Language Model-based Mental Health Chatbots
https://www.researchgate.net/publication/385135480_Improving_Workplace_Well-being_in_Modern_Organizations_A_Review_of_Large_Language_Model-based_Mental_Health_Chatbots